This flowchart, adapted from Dan Abramov's insightful tweet (https://x.com/dan_abramov/status/981712092611989509), illustrates the sequence of events and processes in a React class component's life, from mounting to unmounting, highlighting methods for specific actions at each stage, aligning with React's rendering phases.

- *Mounting*—Initial creation and insertion into the DOM, involving the constructor, initial render, and commit phase that inserts the DOM/refs and calls the `componentDidMount` method.
- *Updating*—Re-rendering due to state or props changes or `forceUpdate()`, with render, pre-commit with `getSnapshotBeforeUpdate` call, and commit phase that updates the DOM/refs and calls the `componentDidUpdate` method. The update can be aborted if `shouldComponentUpdate()` returns false.
- *Unmounting*—Removal from the DOM, skipping the render phase, with the commit phase triggering cleanup, including `componentWillUnmount`.

A.5.2 React functional component lifecycle

React functional components adhere to a lifecycle that mirrors that of class components, but they leverage Hooks to manage the different phases. The `useEffect` hook is the primary mechanism for managing side effects and replicating lifecycle behaviors in functional components. Listing A.4 demonstrates the typical usage of `useEffect`.

Listing A.4 Using the useEffect hook to simulate lifecycle methods

```
const MyComponent = () => {
  useEffect(() => {
    #A
    return () => {
      #B
    };
  }, [dependencies]); #C
  return <div>Hello, World!</div>;
};
```

Table A.11 provides a concise mapping between the `useEffect` hook in